4.5 Prototype Evaluation

Testing is conducted by the developers as the prototype is being built and becomes an important part of prototype evaluation. Testing demonstrates that prototype components are operational, but it's unlikely that test cases will have found all the defects. In the spiral model, the results of evaluation allow the stakeholders and developers to assess whether it is desirable to continue development and create the next prototype. Part of this decision is based on user and stakeholder satisfaction, and part is derived from an assessment of the risks of cost overruns and failure to deliver a working product when the project is finished. Dam and Siang [Dam17] suggest several best-practice tips for gathering feedback on your prototype.

1. Provide scaffolding when asking for prototype feedback.
2. Test your prototype on the right people.
3. Ask the right questions.
4. Be neutral when presenting alternatives to users.
5. Adapt while testing.
6. Allow the user to contribute ideas.

Providing scaffolding is a mechanism for allowing the user to offer feedback that is not confrontational. Users are often reluctant to tell developers that they hate the product they are using. To avoid this, it is often easier to ask the user to provide feedback using a framework such as "I like, I wish, What if" as a means of providing open and honest feedback. *I like* statements encourage users to provide positive feedback on the prototype. *I wish* statements prompt users to share ideas about how the prototype can be improved. These statements can provide negative feedback and constructive criticism. *What if* statements encourage users to suggest ideas for your team to explore when creating prototypes in future iterations.

Getting the right people to evaluate the prototype is essential to reduce the risk of developing the wrong product. Having development team members do all the testing is not wise because they are not likely to be the representative of the intended user population. It's important to have the right mix of users (e.g., novice, typical, and advanced) to give you feedback on the prototype.

Asking the right questions implies that all stakeholders agree on prototype objectives. As a developer, it's important to keep an open mind and do your best to convince users that their feedback is valuable. Feedback drives the prototyping process as you plan for future product development activities. In addition to general feedback, try to ask specific questions about any new features included in the prototype.

Be neutral when presenting alternatives allows the software team to avoid making users feel they are being "sold" on one way to do things. If you want honest feedback, let the users know that you have not already made up your mind that there is only one right way to do things. *Egoless programming* is a development philosophy that focuses on producing the best product the team can create for the intended users. Although it is not desirable to create throwaway prototypes, egoless programming suggests that things that are not working need to be fixed or

discarded. So try not to become too attached to your ideas when creating early prototypes.

Adapt while testing means that you need a flexible mind-set while users are working with the prototype. This might mean altering your test plan or making quick changes to the prototype and then restarting the testing. The goal is to get the best feedback you can from users, including direct observation of them as they interact with the prototype. The important thing is that you get the feedback you need to help decide whether to build the next prototype or not.

Allow users to contribute ideas means what it says. Make sure you have a way of recording their suggestions and questions (electronically or otherwise). We will discuss additional ways of conducting user testing in Chapter 12.

SafeHome



Room Designer
Evaluating First Prototype

The scene: Doug Miller's office.

The players: Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member.

The conversation: (A knock on the door. Jamie and Vinod enter Doug's office.)

Jamie: Doug, you got a minute?

Doug: Sure, Jamie, what's up?

Jamie: We completed the evaluation of the *SafeHome* room design tool working with our marketing stakeholders.

Doug: How did things go?

Vinod: We mostly focused on the user interface that will allow users to place alarm sensors in the room.

Jamie: I am glad we let them review a paper prototype before we created the PC prototype.

Doug: Why is that?

Vinod: We made some changes, and the marketing people liked the new design better, so that's the design we used when we started programming it.

Doug: Good. What's the next step?

Jamie: We completed the risk analysis and since we did not pick up any new user stories, we think it is reasonable to work on creating the next incremental prototype since we are still on time and within budget.

Vinod: So, if you agree, we'll get the developers and stakeholders together and begin planning the next software increment.

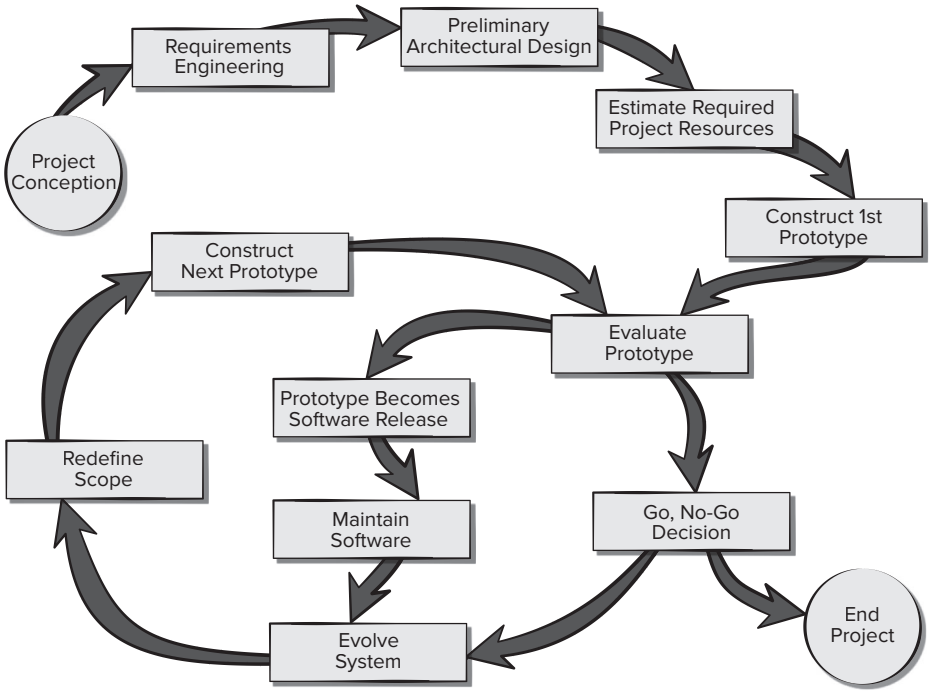
Doug: I agree. Keep me in the loop and try to keep the development time on the next prototype to 6 weeks or less.

4.6 Go, No-Go Decision

After the prototype is evaluated, project stakeholders decide whether to continue development of the software product. If you refer to Figure 4.4, a slightly different decision based on the prototype evaluation might be to release the prototype to the

FIGURE 4.4

Recommended software process model



end users and begin the maintenance process. Sections 4.8 and 4.9 discuss this decision in more detail. We discussed the use of risk assessment in the spiral model as part of the prototype assessment process in Section 2.5.3. The first circuit around the spiral might be used to solidify project requirements. But it really should be more. In the method we are proposing here, each trip around the spiral develops a meaningful increment of the final software product. You can work with the project user story or feature backlog to identify an important subset of the final product to include in the first prototype and repeat this for each subsequent prototype.

A pass through the planning region follows the evaluation process. Revised cost estimates and schedule changes are proposed based on what was discovered when evaluating the current software prototype. This may involve adding new user stories or features to the project backlog as the prototype is evaluated. The risk of exceeding the budget and missing the project delivery date is assessed by comparing new cost and time estimates against old ones. The risk of failing to satisfy user expectations is also considered and discussed with the stakeholders and sometimes senior management before deciding to create another prototype.

The goal of the risk assessment process is to get the commitment of all stakeholders and company management to provide the resources needed to create the next prototype. If the commitment is not there because the risk of project failure is too great, then the project can be terminated. In the Scrum framework (Section 3.4), the go, no-go decision might be made during the Scrum retrospective

meeting held between the prototype demonstration and the new sprint planning meeting. In all cases, the development team lays out the case for the product owners and lets them decide whether to continue product development or not. A more detailed discussion of software risk assessment methods is presented in Chapter 26.

4.7 PROTOTYPE EVOLUTION

Once a prototype has been developed and reviewed by the development team and other stakeholders, it's time to consider the development of the next prototype. The first step is to collect all feedback and data from the evaluation of the current prototype. The developers and stakeholders then begin negotiations to plan the creation of another prototype. Once the desired features for the new prototype have been agreed upon, consideration is given to any known time and budget constraints as well as the technical feasibility of implementing the prototype. If the development risks are deemed to be acceptable, the work continues.

The evolutionary prototyping process model is used to accommodate changes that inevitably occur as software is developed. Each prototype should be designed to allow for future changes to avoid throwing it away and creating the next prototype from scratch. This suggests favoring both well-understood and important features when setting the goals for each prototype. As always, the customer's needs should be given great importance in this process.

4.7.1 New Prototype Scope

The process of determining the scope of a new prototype is like the process of determining the scope of the initial prototype. Developers would either: (1) select features to develop within the time allocated to a sprint, or (2) allocate sufficient time to implement the features needed to satisfy the goals set by the developers with stakeholder input. Either approach requires the developers to maintain a prioritized list of features or user stories. The priorities used to order the list should be determined by the goals set for the prototype by the stakeholders and developers.

In XP (Section 3.5.1), the stakeholders and developers work together to group the most important user stories into a prototype that will become the next release of the software and determine its completion date. In Kanban (Section 3.5.2), developers and stakeholders make use of a board that allows them to focus on the completion status of each user story. This is a visual reference that can be used to assist developers using any incremental prototype process model to plan and monitor the progress of the software development. Stakeholders can easily see the feature backlog and help the developers order it to identify the most useful stories needed in the next prototype. It is probably easier to estimate the time required to complete the selected user stories than to find the user stories that need to fit into a fixed time block. But the advice to keep prototype development time to 4 to 6 weeks should be followed to ensure adequate stakeholder involvement and feedback.

4.7.2 Constructing New Prototypes

A user story should contain both a description of how the customer plans to interact with the system to achieve a specific goal and a description of what the customer's definition of acceptance is. The development team's task is to create additional software components to implement the user stories selected for inclusion in the new prototype along with the necessary test cases. Developers need to continue communication with all stakeholders as they create the new prototype.

What makes this new prototype trickier to build is that software components created to implement new features in the evolving prototype need to work with the components used to implement the features included in the previous prototype. It gets even trickier if developers need to remove components or modify those that were included in the previous prototype because requirements have changed. Strategies for managing these types of software changes are discussed in Chapter 22.

It is important for the developers to make design decisions that will make the software prototype more easily extensible in the future. Developers need to document design decisions in a way that will make it easier to understand the software when making the next prototype. The goal is to be agile in both development and documentation. Developers need to resist the temptation to overdesign the software to accommodate features that may or may not be included in the final product. They also should limit their documentation to that which they need to refer to during development or when changes need to be made in the future.

4.7.3 Testing New Prototypes

Testing the new prototype should be relatively straightforward if the development team created test cases as part of the design process before the programming was completed. Each user story should have had acceptance criteria attached to it as it was created. These acceptance statements should guide the creation of the test cases intended to help verify that the prototype meets customer needs. The prototype will need to be tested for defects and performance issues as well.

One additional testing concern for evolutionary prototypes is to ensure that adding new features does not accidentally break features that were working correctly in the previous prototype. *Regression testing* is the process of verifying that software that was previously developed and tested still performs the same way after it has been changed. It is important to use your testing time wisely and make use of the test cases that are designed to detect defects in the components most likely to be affected by the new features. Regression testing is discussed in more detail in Chapter 20.

4.8 Prototype Release

When an evolutionary prototyping process is applied, it can be difficult for developers to know when a product is finished and ready for release to the customers. Software developers do not want to release a buggy software product to the end users and have them decide the software has poor quality. A prototype being considered as a release candidate must be subjected to user acceptance testing in addition to functional and nonfunctional (performance) testing that would have been conducted during prototype construction.

User acceptance tests are based on the agreed-upon acceptance criteria that were recorded as each user story was created and added to the product backlog. This allows

user representatives to verify that the software behaves as expected and collect suggestions for future improvements. David Nielsen [Nie10] has several suggestions for conducting prototype testing in industrial settings.

When testing a release candidate, functional and nonfunctional tests should be repeated using the test cases that were developed during the construction phases of the incremental prototypes. Additional nonfunctional tests should be created to verify that the performance of the prototype is consistent with the agreed-upon benchmarks for the final product. Typical performance benchmarks may deal with system response time, data capacity, or usability. One of the most important nonfunctional requirements to verify is ensuring that the release candidate will run in all planned run-time environments and on all targeted devices. The process should be focused on testing limited to the acceptance criteria established before the prototype was created. Testing cannot prove a software product is bug free, only that the test cases ran correctly.

User feedback during acceptance testing should be organized by user-visible functions as portrayed via the user interface. Developers should examine the device in question and make changes to the user interface screen if implementing these changes will not delay the release of the prototype. If changes are made, they need to be verified in a second round of testing before moving on. You should not plan for more than two iterations of user acceptance testing.

It is important, even for projects using agile process models, to use an issue tracking or bug reporting system (e.g., Bugzilla[2] or Jira[3]) to capture the testing results. This allows developers to record test failures and makes it easier to identify the test cases that will need to be run again to verify that a repair properly corrects the problem that was uncovered. In each case the developers need to assess whether the changes can be made to the software without causing a cost overrun or late product delivery. The implications of not fixing a problem need to be documented and shared with both the customer and senior managers who may decide to cancel the project rather than committing the resources needed to deliver the final project.

The issues and lessons learned from creating the release candidate should be documented and considered by the developers and stakeholders as part of the project postmortem. This information should be considered before deciding to undertake future development of a software product following its release to the user community. The lessons learned from the current product can help developers make better cost and time estimates for similar projects in the future.

Techniques for conducting user acceptance testing are discussed in Chapters 12 and 20. A more detailed discussion on software quality assurance is presented in Chapter 17.

4.9 Maintain Release Software

Maintenance is defined as the activities needed to keep software operational after it has been accepted and delivered (released) in the end-user environment. Maintenance will continue for the life of the software product. Some software engineers believe that the majority of the money spent on a software product will be spent on maintenance activities. *Corrective maintenance* is the reactive modification of software to repair problems

2 https://www.bugzilla.org/.
3 https://www.atlassian.com/software/jira.